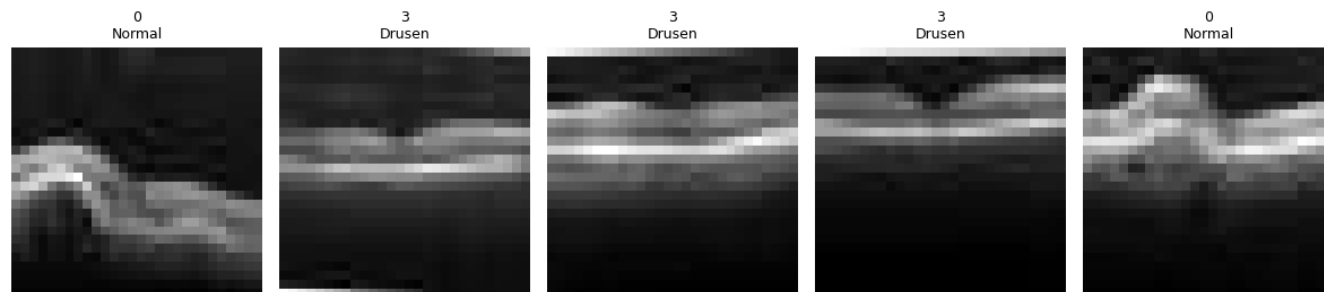


BME 4801:OCT Dataset Project Report

Presented by: Himanshu Thakur and Samuel Schultz

Problem & Dataset

- Optical Coherence Tomography (OCT) is an imaging technique that generates a picture of the retina by measuring the amount of dim red light reflected off the retina and optic nerve
 - Used to measure retina thickness and diagnose diseases/conditions
 - Choroidal neovascularization
 - Diabetic macular edema
 - Drusen
- OCT-MNIST dataset has preprocessed images (28x28 arrays) separated into three subsets (training, validation, test) with classification labels



Project Deliverables

- One model must combine >2 classifiers from the Scikit-Learn library
- One model must have at least one artificial neural network
- Each model must be selected from >3 preliminary model designs through screening
 - Models past the screen process must be optimized
- Model results must be provided on test data

Testing Procedure

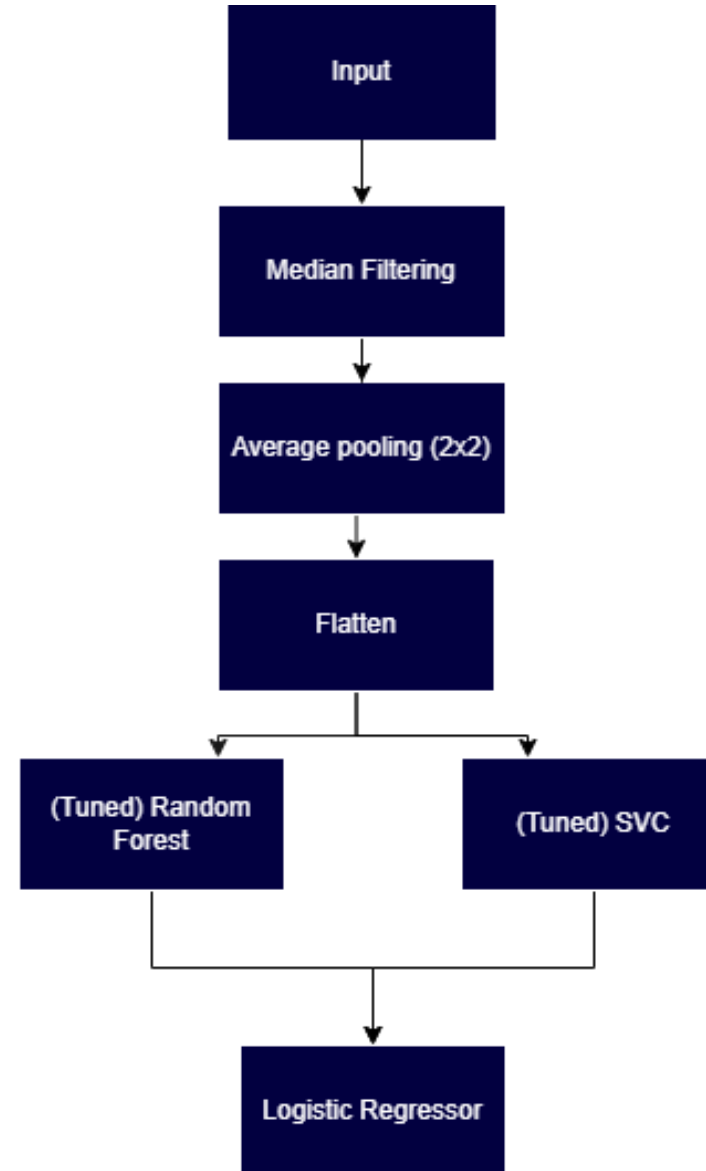
- Train shape: (97477, 28, 28)
- Val shape: (10832, 28, 28)
- Test shape: (1000, 28, 28)
- 30,000 images from training were used
- All 1,000 images in the test set were used
- Metrics: accuracy, precision, recall, ROC, AUC

Preprocessing Initial Ideas

Pipeline	Input	Number Features	Train Time (s)	Val Acc	Val Precision	Val Recall
Baseline	28x28 raw pixels	784	706.322	0.6495	0.5995	0.6495
AvgPool (28 --> 14)	Pooled pixels	196	147.204	0.6515	0.6814	0.6515
AvgPool (28 --> 7)	Pooled pixels	49	21.403	0.6365	0.6417	0.6365
Median (3x3) + AvgPool (28 --> 14)	Filtered + Pooled pixels	196	405.530	0.6560	0.6910	0.6560
MaxPool (28 --> 14)	Pooled pixels	196	122.338	0.6365	0.6812	0.6365
PCA (50)	PCA pixels	50	4.448	0.5970	0.6745	0.5970
Generated Features	mean/std/min/max/median/Q1/Q3	7	17.368	0.5415	0.6267	0.5415
Median then PCA	Filtered + PCA Pixels	200	433.93	0.606	0.5461	0.606

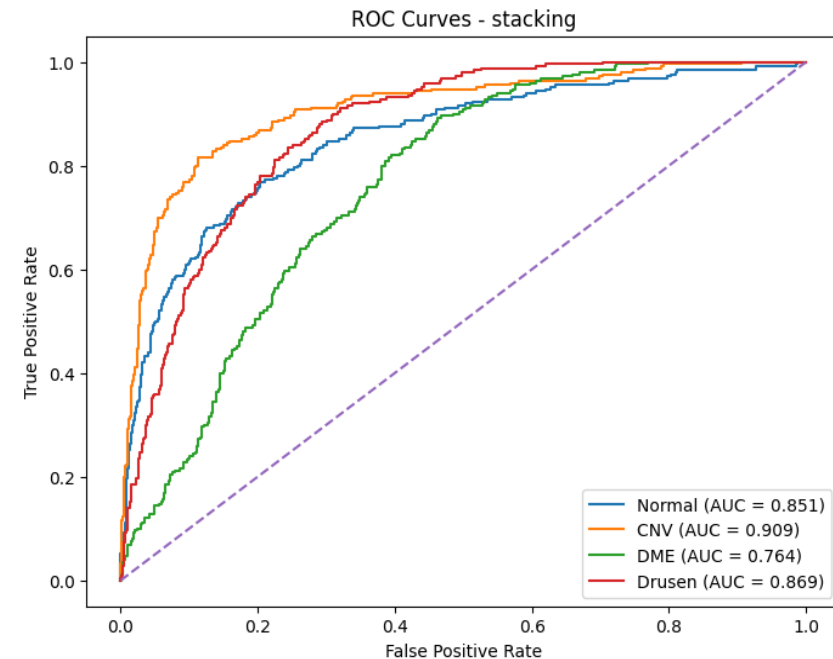
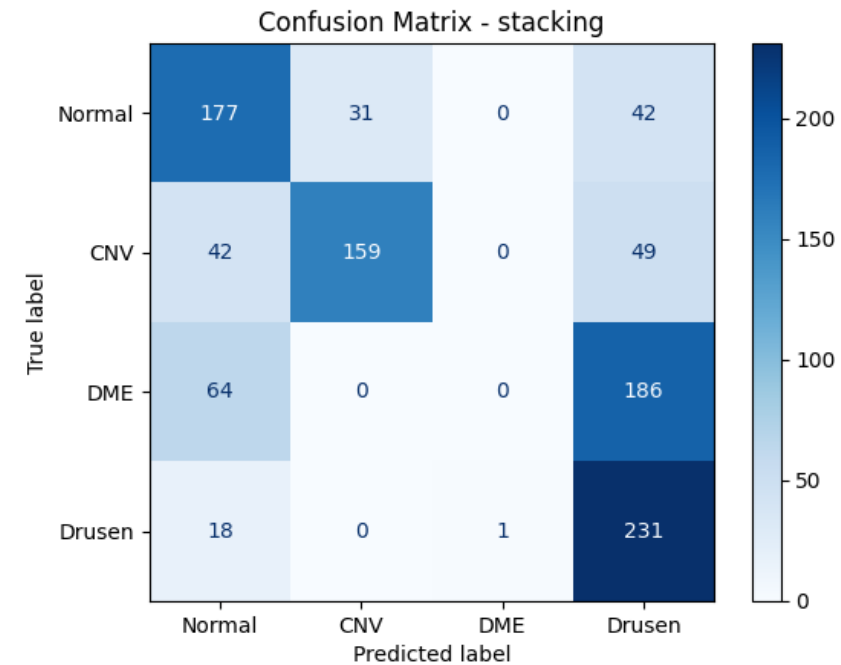
Model 1: Preprocessed Stacking

- Preprocessing performed on both classifier models
 - Median filtering (3x3)
 - Average pooling (2x2)
 - Flatten data
- Stacking classifier
 - Random forest (25% max features, 1 min sample per leaf, 200 estimators)
 - SVC (rbf kernel)
- Outputs combined using Logistic Regression as the final estimator



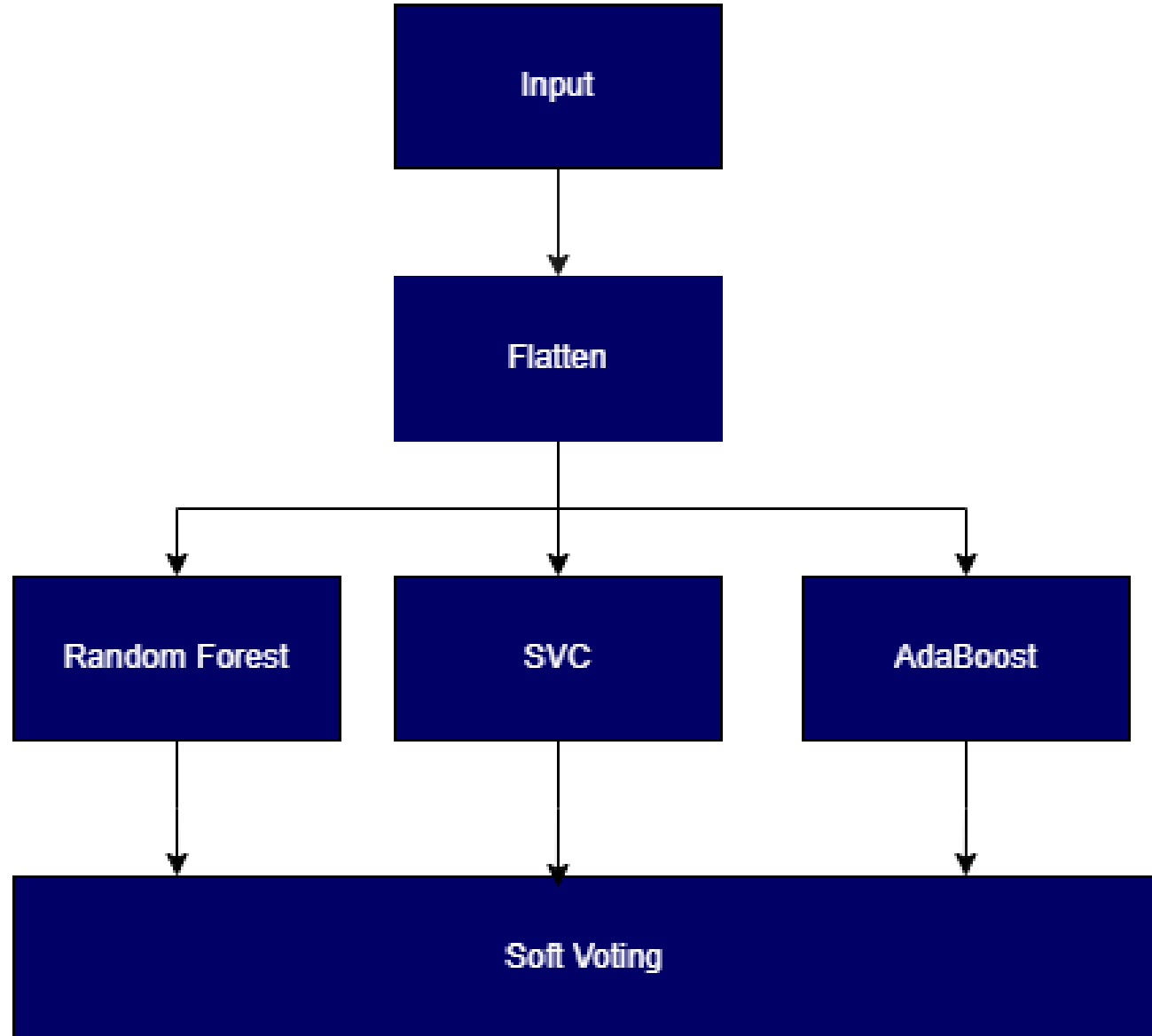
Model 1 Test Results

- Cross validation
 - Accuracy: 0.7250
 - Precision: 0.7284
 - Recall: 0.7250
- Test
 - Accuracy: 0.5670
 - Precision: 0.4699
 - Recall: 0.5670
- Training Time
 - 10 min 14 seconds



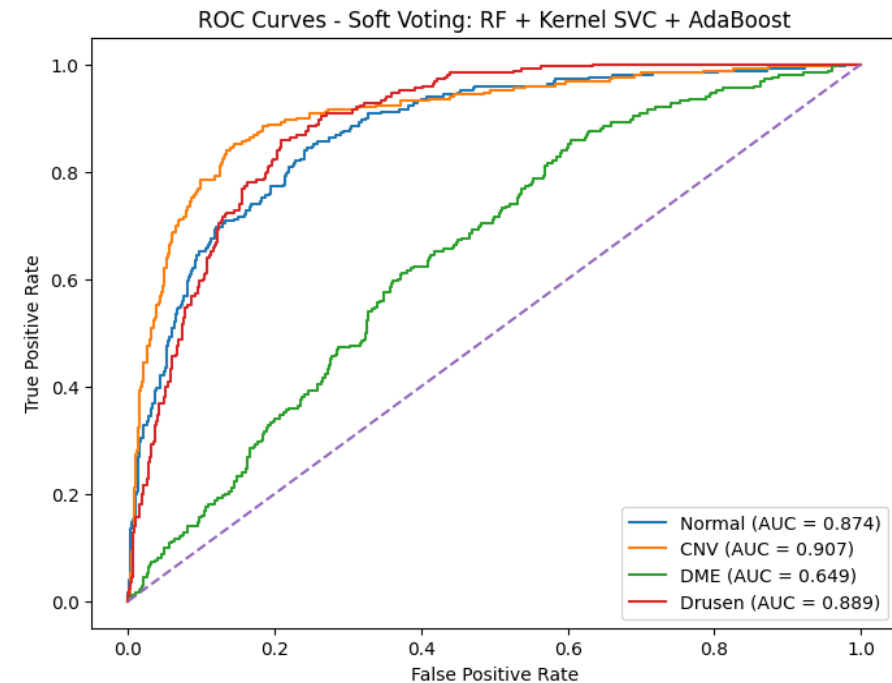
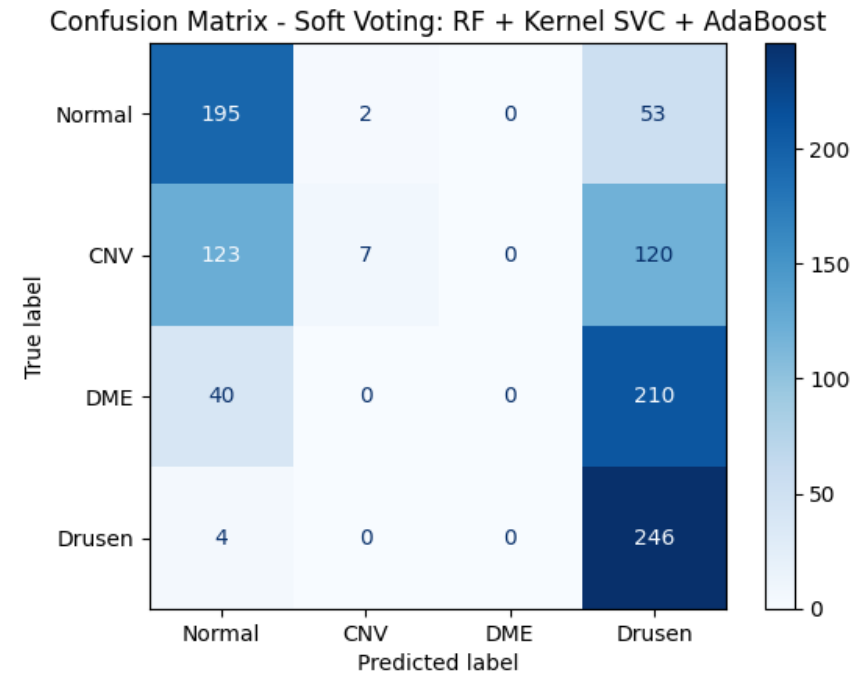
Model 2: Voting Classifier

- More minimal preprocessing
 - Flatten data
- Soft Voting classifier
 - Random forest (25% max features, 1 min sample per leaf, 200 estimators)
 - SVC (rbf kernel)
 - Ada boost (Decision Tree Stumps)
- Soft voting so averages predicted probabilities from multiple base models



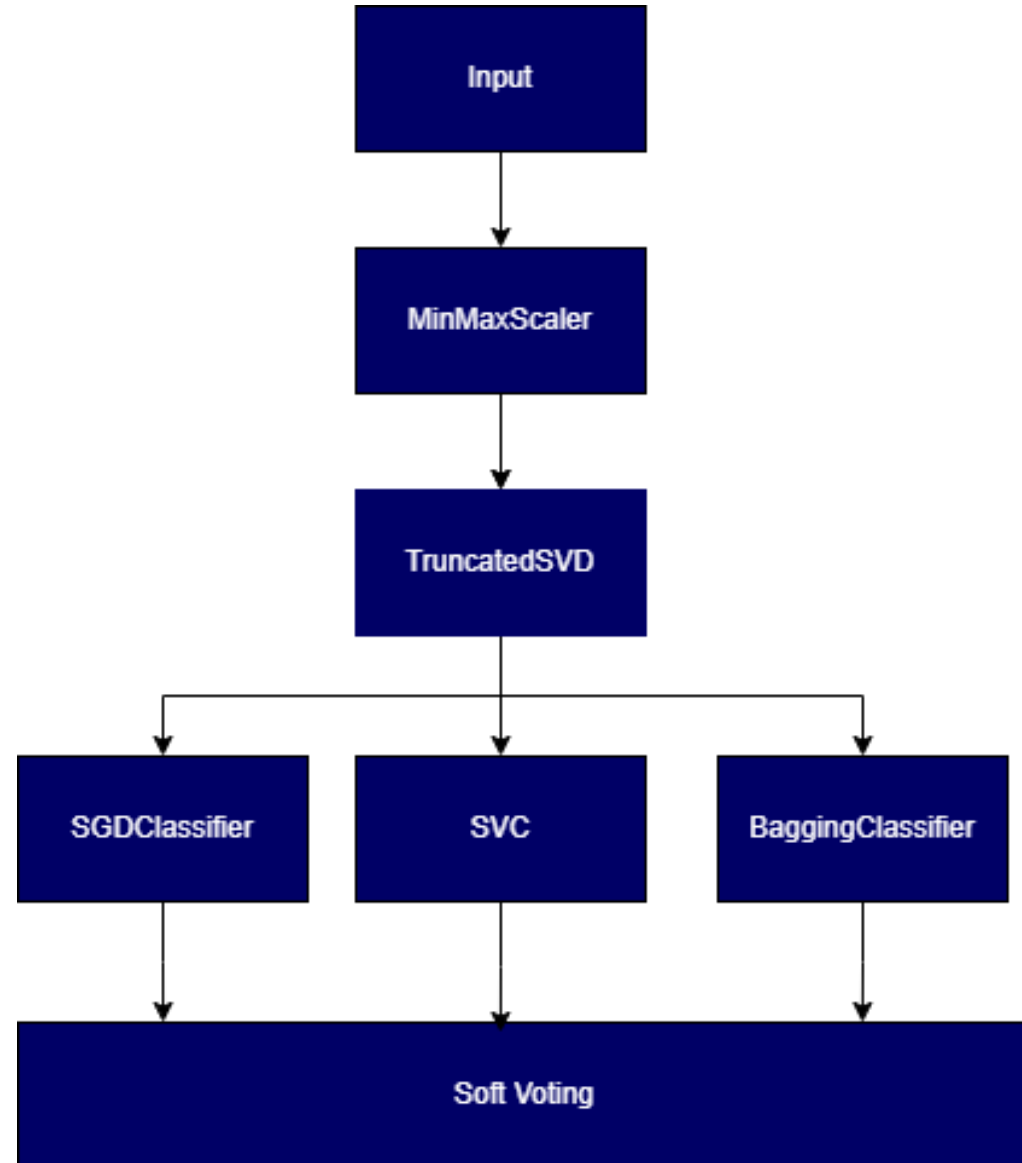
Model 2 Test Results

- Test
 - Accuracy: 0.448
 - Precision: 0.6769
 - Recall: 0.448
- Training Time
 - 3 min 56 seconds



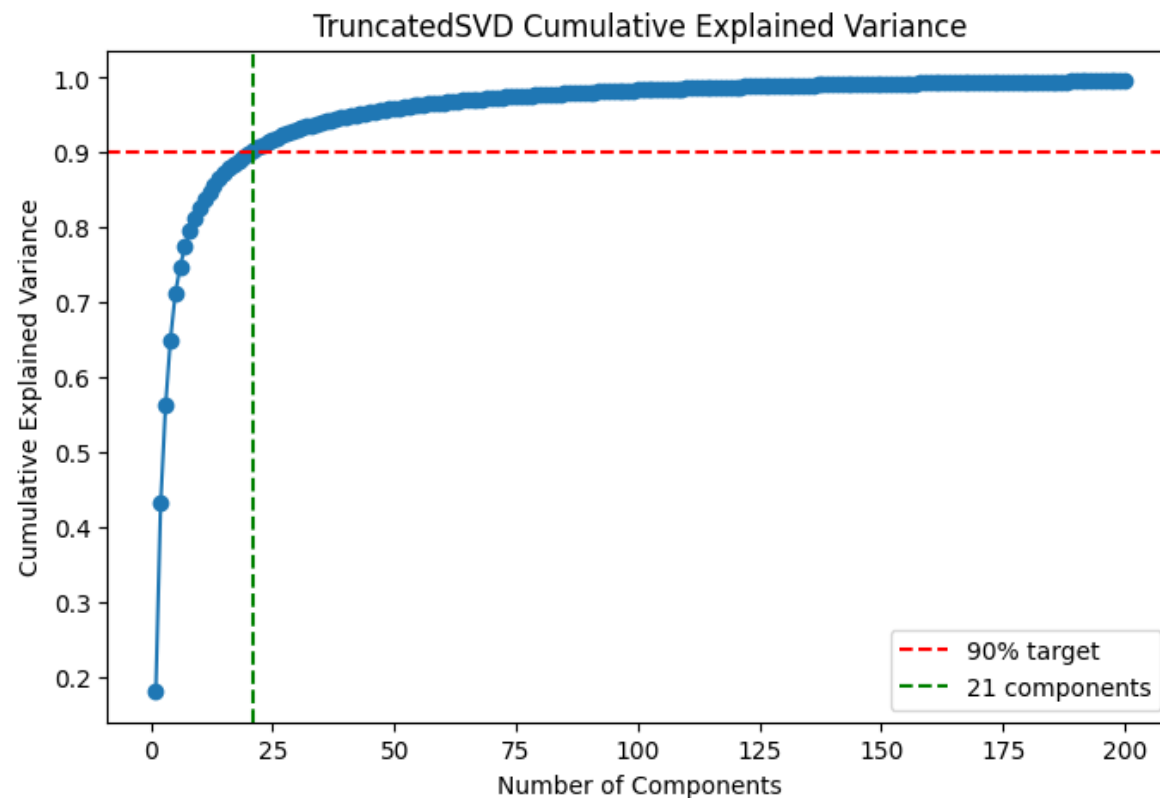
Model 3: Voting Classifier

- Preprocessing
 - Flatten data
 - Scale (MinMaxScaler)
 - TruncatedSVD($n_components=21$)
 - Doesn't need to have centered data (mean subtracted) as it operates directly on the raw data matrix
- Soft voting classifier
 - SVC(kernel="rbf", C=1.0)
 - SGDClassifier(loss="log_loss")
 - BaggingClassifier(estimator=DecisionTreeClassifier(), n_estimators=50, max_samples=0.8, ...)



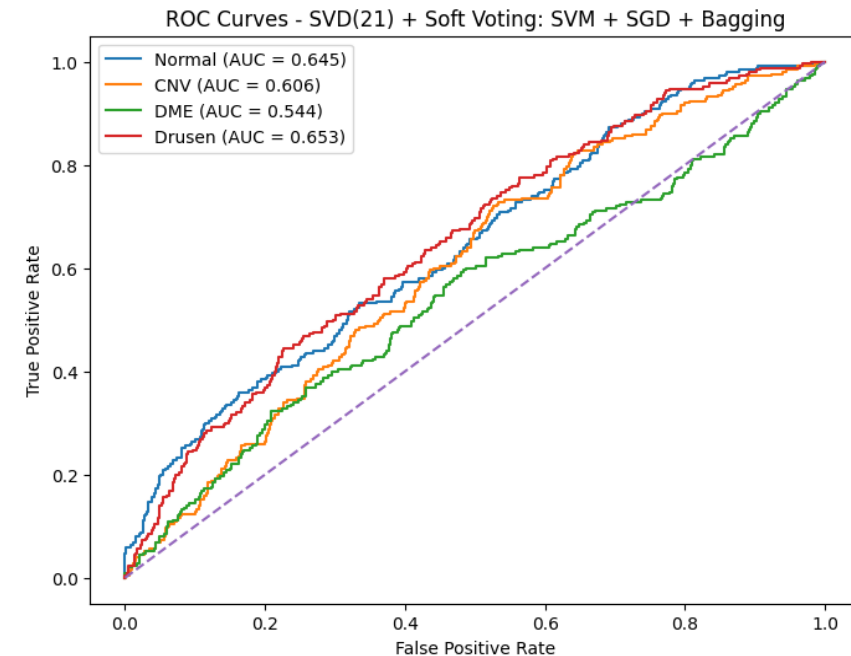
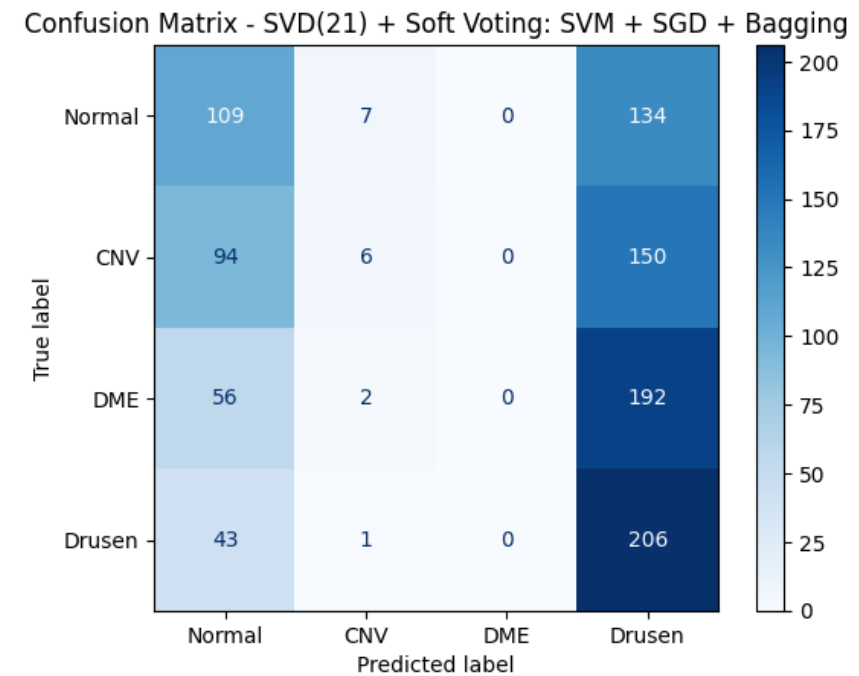
Model 3: TruncatedSVD

- To find the number of components, we used the elbow plot method to find how many components preserve 90% of the variance



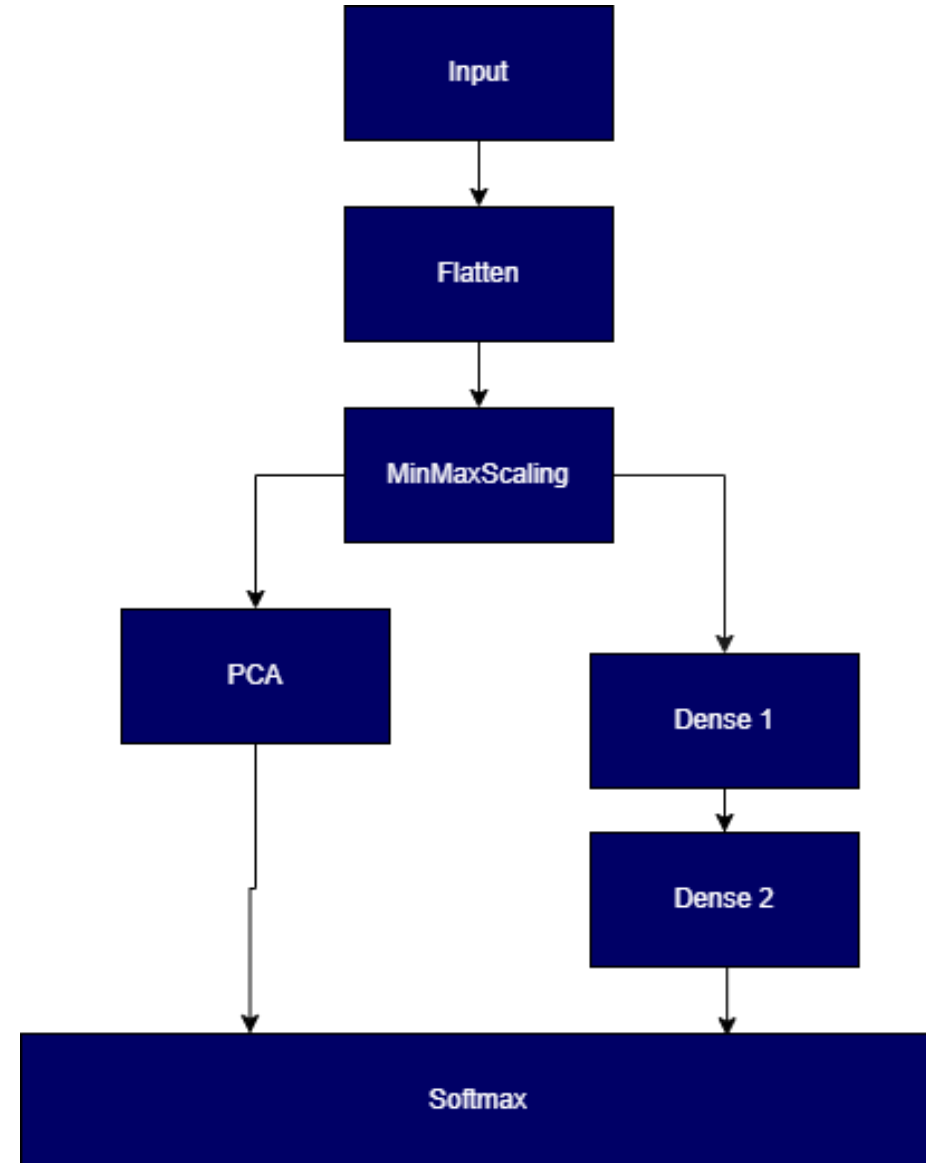
Model 3 Test Results

- Cross validation
 - Accuracy: 0.5160
 - Precision: 0.4894
 - Recall: 0.5160
- Test
 - Accuracy: 0.3210
 - Precision: 0.5095
 - Recall: 0.3210
- Training Time
 - 2 minutes 39 seconds



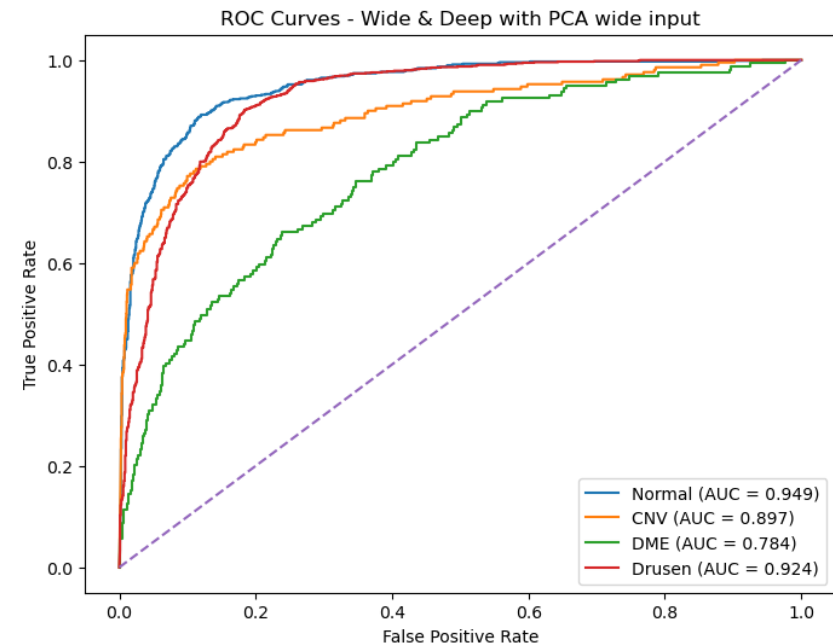
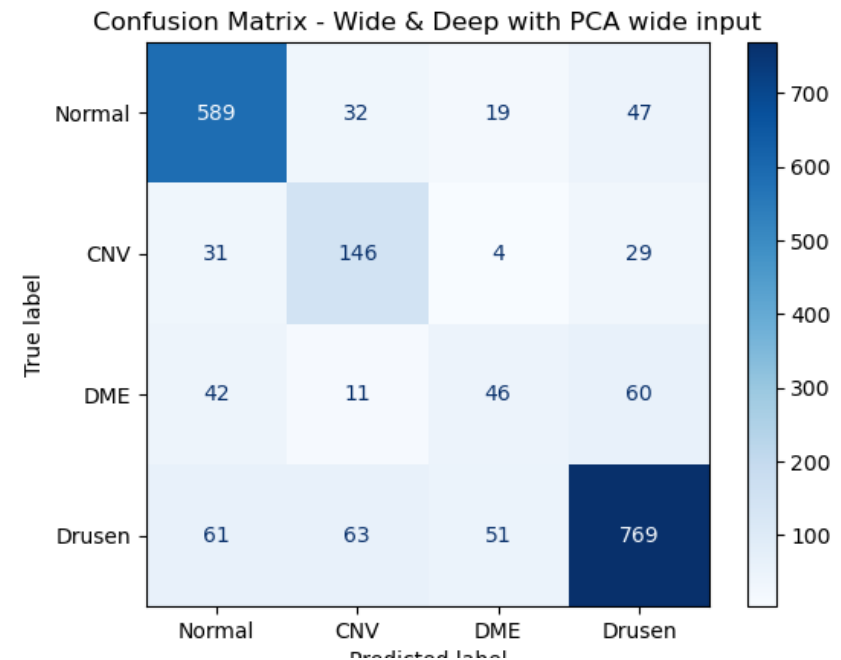
Model 4: Wide & Deep Neural Network

- Preprocessing:
 - Scaled inputs for the deep path
 - Scaled and PCA reduced (100 components) for the wide path
 - PCA for 95% variance: 42, PCA for 99% variance 133
- ANN with two paths:
 - One 'Deep' Path with a two layers layers of 300 nodes each with Relu Activation
 - One 'Wide' Path where the dense layer are skipped
 - Both paths concatenate outputs at the end and are fed to a Softmax layer

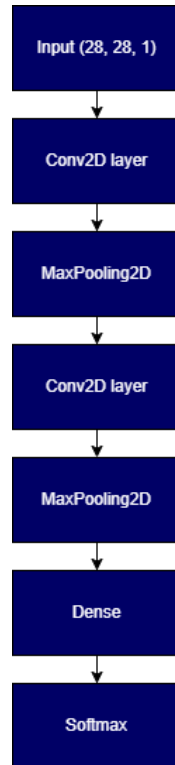


Model 4 Test Results

- Cross validation
 - Accuracy: 0.7719
 - Precision: 0.7686
 - Recall: 0.7719
 - Training time: 6 min 58 seconds (CPU)
- Test
 - Accuracy: 0.7750
 - Precision: 0.7720
 - Recall: 0.7750
 - AUC Macro: 0.8885
 - AUC Weighted: 0.9186



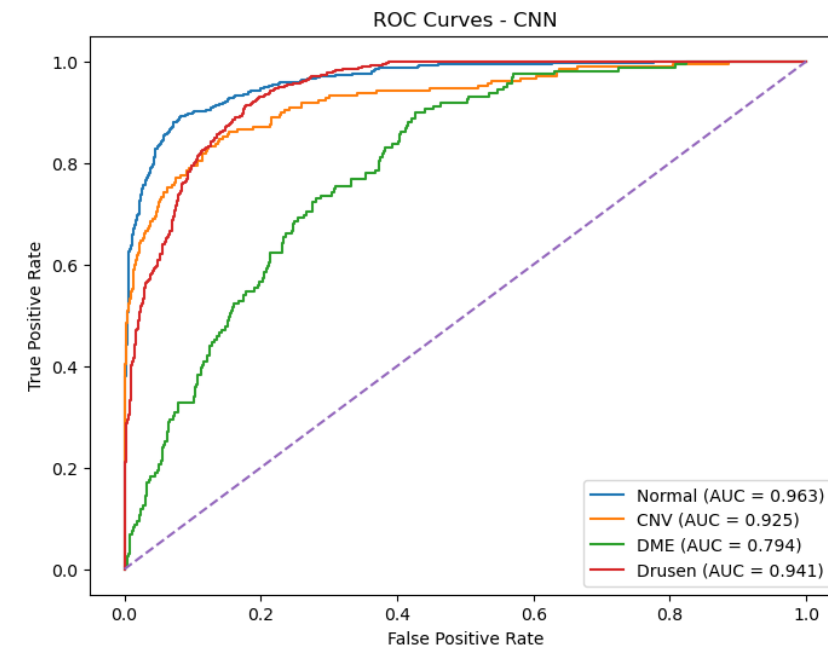
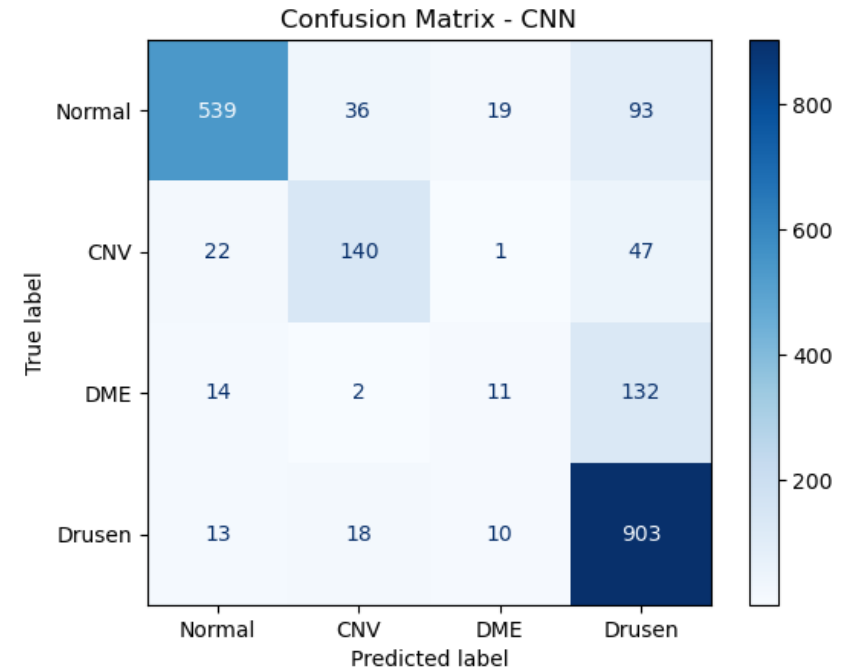
Model 5: CNN



- Preprocessing
 - Normalized
 - Starts with original raw data with added dimension
- CNN architecture
 - Input layer (28, 28, 1)
 - Conv2D layer (64, kernel_size=3, relu)
 - MaxPooling2D(pool_size=2)
 - Conv2D and Pooling repeated
 - Flatten
 - Dense layer (32 neurons, relu)
 - Dense layer (4 neurons, softmax)

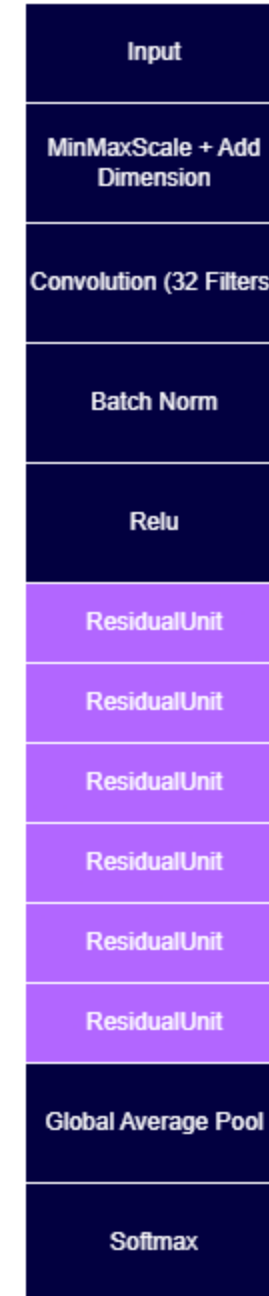
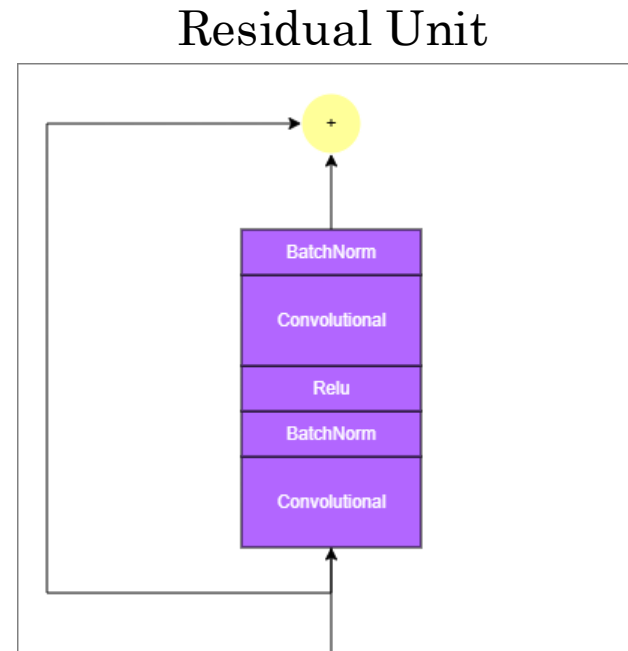
Model 5 Test Results

- Cross validation
 - Accuracy: 0.7869
 - Precision: 0.7606
 - Recall: 0.7869
 - Training time: 6 min 58 seconds (CPU)
- Test
 - Accuracy: 0.7965
 - Precision: 0.7740
 - Recall: 0.7965
 - AUC Macro: 0.9056
 - AUC Weighted: 0.9349



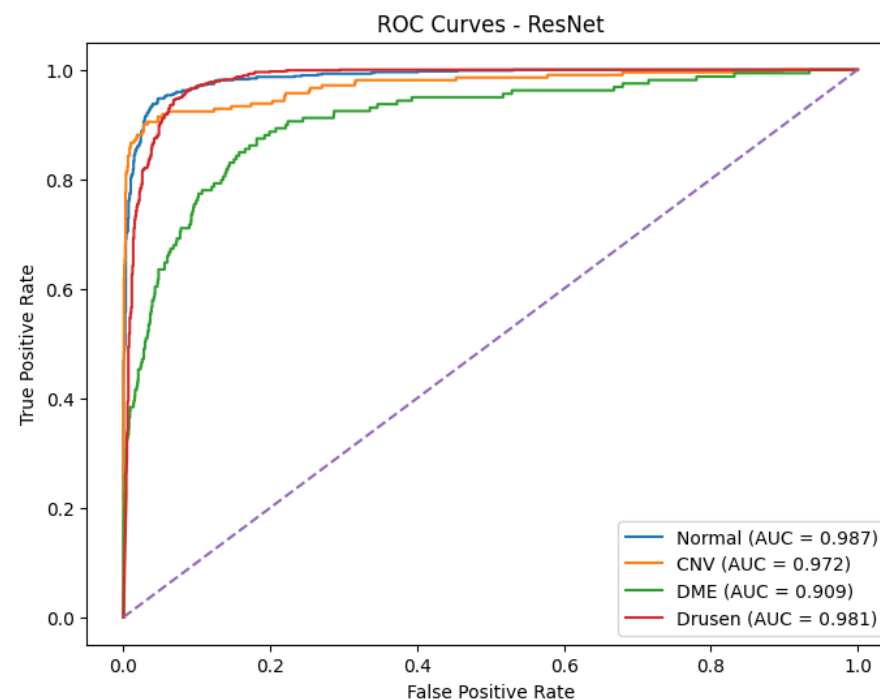
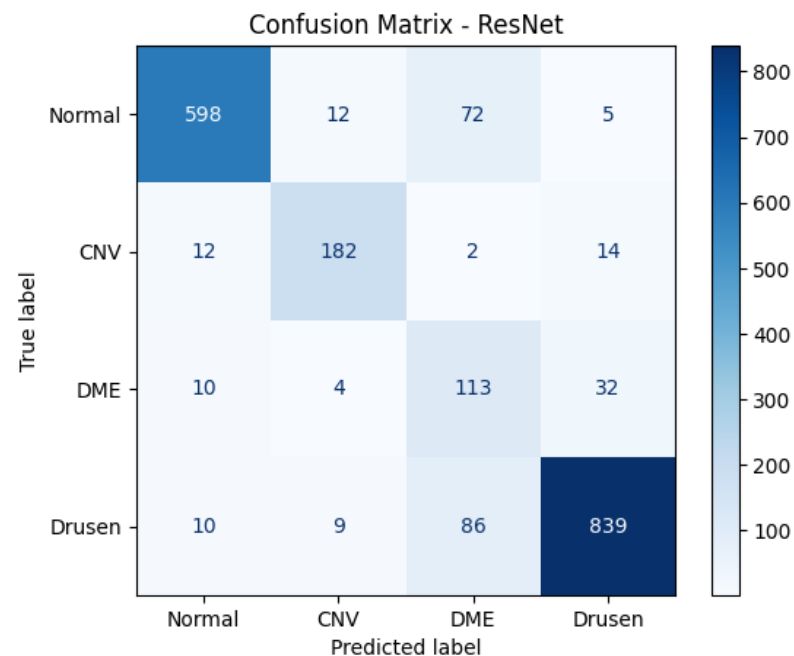
Model 6: Resnet Architecture

- Motivation:
 - By modeling $h(x) = F(x) + x$. We allow for much deeper architectures in theory
- Architecture:
 - A Single Residual Unit consists of: 1 Convolutional layer + Batch norm, 1 Relu activation layer, then another Convolutional + Batch Norm. The inputs to the layer are then concatenated at the end.
 - Full Architecture:
 - MinMaxScaler + Add dimension
 - Convolution (32 Filters, strides=1)
 - BatchNorm
 - Relu
 - ResidualUnit (32 Filters, strides=1)
 - ResidualUnit(32 Filters, strides=1)
 - ResidualUnit(64 Filters, strides=2)
 - ResidualUnit(64 Filters, strides=1)
 - ResidualUnit(128 Filters, strides=2)
 - ResidualUnit(128 Filters, strides=1)
 - GlobalAvgpool
 - Softmax



Model 6 Test Results

- Cross validation
 - Accuracy: 0.8899
 - Precision: 0.8912
 - Recall: 0.8899
 - Time: 5 Minutes 39s (GPU)
- Test
 - Accuracy: 0.886
 - Precision: 0.8962
 - Recall: 0.886
 - AUC Macro: 0.8885
 - AUC Weighted: 0.9186



Results Summary

	Accuracy	Precision	Recall	Training Time (min)
Model 1 (Preprocessed Stack)	0.567	0.4699	0.567	10.14
Model 2 (Voting Classifier)	0.448	0.6769	0.448	3.56
Model 3 (Truncated Voting)	0.321	0.5095	0.3210	2.39
Model 4 (Wide & Deep)	0.7750	0.772	0.775	6.58
Model 5 (CNN)	0.7965	0.774	0.7965	5.41
Model 6 (ResNet)	0.886	0.89	0.899	5.66 (GPU)

Conclusion:

- In conclusion: Traditional CNN + Resnet Architecture show the most promising results. With Wide and Deep Network not being too far behind.
- Resnet has clearly the strongest performance, however it takes LONG to train. If dealing with massive quantities of Data a traditional CNN or Wide and Deep Network may be recommended otherwise Resnet is the clear winner.
- For Scikit learn model, we would choose model 1